In [96]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.ensemble import RandomForestClassifier
```

In [97]:
```
df = pd.read_csv(r'C:\Users\NIKITA\Downloads\capstone_500.csv', encoding='ISO-8859-1')
```

In [98]:
```
df.columns = df.columns.str.strip()
```

In [99]:
```
df.head()
```

Out[99]:

	Age	City	Occupation	Which subscription have you used?	How long have you been subscribed?	How often do you order food?	Approx monthly spend on food delivery	Rate your satisfaction with delivery experience	Rate your satisfaction with restaurant options	Rate your satisfaction with app experience	Rate value for money	Have yo cancelle you subscriptio
0	21	Thane	Student	Zomato Gold	More than 1 year	Weekly	1500□3000	4	3	4	5	N
1	27	Thane	Working Professional	Zomato Gold	More than 1 year	Weekly	0□500	4	4	4	4	N
2	21	Ulhasnagar	Student+ working	Both	More than 1 year	2□3 times a week	1500□3000	5	5	4	1	N
3	21	Mumbai	Student	Zomato Gold	Less than 3 months	Weekly	500□1500	4	5	4	3	N
4	21	Mumbai	Student	NaN	Less than 3 months	Weekly	500□1500	4	4	4	3	N

In [100…

```
print(df.columns.tolist())
```

```
['Age', 'City', 'Occupation', 'Which subscription have you used?', 'How long have you been subscribed?', 'How often do you orde
r food?', 'Approx monthly spend on food delivery', 'Rate your satisfaction with delivery experience', 'Rate your satisfaction w
ith restaurant options', 'Rate your satisfaction with app experience', 'Rate value for money', 'Have you cancelled your subscri
ption?', 'Are you planning to cancel?', 'If yes, what is the main reason?', 'How likely are you to recommend this subscriptio
n?', 'Would you renew your subscription?']
```

In [101…

```
df.rename(columns={
    'Age': 'Age',
    'City': 'City',
    'Occupation': 'Occupation',
    'Which subscription have you used?': 'Subscription_Type',
    'How long have you been subscribed?': 'Subscription_Duration',
    'How often do you order food?': 'Order_Frequency',
```

```
    'Approx monthly spend on food delivery': 'Monthly_Spend',
    'Rate your satisfaction with delivery experience': 'Delivery_Rating',
    'Rate your satisfaction with restaurant options': 'Restaurant_Rating',
    'Rate your satisfaction with app experience': 'App_Rating',
    'Rate value for money': 'Value_Rating',
    'Have you cancelled your subscription?': 'Cancelled',
    'Are you planning to cancel?': 'Planning_To_Cancel',
    'If yes, what is the main reason?': 'Cancellation_Reason',
    'How likely are you to recommend this subscription?': 'Recommendation_Score',
    'Would you renew your subscription?': 'Renew_Subscription'
}, inplace=True)
```

In [102…

```
print(df.columns.tolist())
```

```
['Age', 'City', 'Occupation', 'Subscription_Type', 'Subscription_Duration', 'Order_Frequency', 'Monthly_Spend', 'Delivery_Rating', 'Restaurant_Rating', 'App_Rating', 'Value_Rating', 'Cancelled', 'Planning_To_Cancel', 'Cancellation_Reason', 'Recommendation_Score', 'Renew_Subscription']
```

In [103…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 16 columns):
 #   Column                 Non-Null Count  Dtype 
---  ------                 --------------  ----- 
 0   Age                    500 non-null    int64 
 1   City                   500 non-null    object
 2   Occupation             500 non-null    object
 3   Subscription_Type      400 non-null    object
 4   Subscription_Duration  500 non-null    object
 5   Order_Frequency        500 non-null    object
 6   Monthly_Spend          500 non-null    object
 7   Delivery_Rating        500 non-null    int64 
 8   Restaurant_Rating      500 non-null    int64 
 9   App_Rating             500 non-null    int64 
 10  Value_Rating           500 non-null    int64 
 11  Cancelled              500 non-null    object
 12  Planning_To_Cancel     500 non-null    object
 13  Cancellation_Reason    255 non-null    object
 14  Recommendation_Score   500 non-null    int64 
 15  Renew_Subscription     500 non-null    object
dtypes: int64(6), object(10)
memory usage: 62.6+ KB
```

In [104…

```
df.isnull().sum()
```

Out[104…

```
Age                       0
City                      0
Occupation                0
Subscription_Type       100
Subscription_Duration     0
Order_Frequency           0
Monthly_Spend             0
Delivery_Rating           0
Restaurant_Rating         0
App_Rating                0
Value_Rating              0
Cancelled                 0
Planning_To_Cancel        0
Cancellation_Reason     245
Recommendation_Score      0
Renew_Subscription        0
dtype: int64
```

In [105…

```
print(df[['Subscription_Type', 'Cancellation_Reason']].isnull().sum())
```

```
Subscription_Type      100
Cancellation_Reason    245
dtype: int64
```

In [106…

```
df['Subscription_Type'] = df['Subscription_Type'].fillna('Not Subscribed')

df['Cancellation_Reason'] = df['Cancellation_Reason'].fillna('No Reason')
```

In [107…

```
df.isnull().sum()
```

Out[107…
```
Age                       0
City                      0
Occupation                0
Subscription_Type         0
Subscription_Duration     0
Order_Frequency           0
Monthly_Spend             0
Delivery_Rating           0
Restaurant_Rating         0
App_Rating                0
Value_Rating              0
Cancelled                 0
Planning_To_Cancel        0
Cancellation_Reason       0
Recommendation_Score      0
Renew_Subscription        0
dtype: int64
```

In [108…
```
print(df[['Subscription_Type', 'Cancellation_Reason']].isnull().sum())
```

```
Subscription_Type      0
Cancellation_Reason    0
dtype: int64
```

In [109…
```
df['Monthly_Spend'] = df['Monthly_Spend'].astype(str)

df['Monthly_Spend'] = df['Monthly_Spend'].str.extract(r'(\d+)')

df['Monthly_Spend'] = pd.to_numeric(df['Monthly_Spend'], errors='coerce')
```

In [110…
```
df['Monthly_Spend'].dtype
```

Out[110…
```
dtype('int64')
```

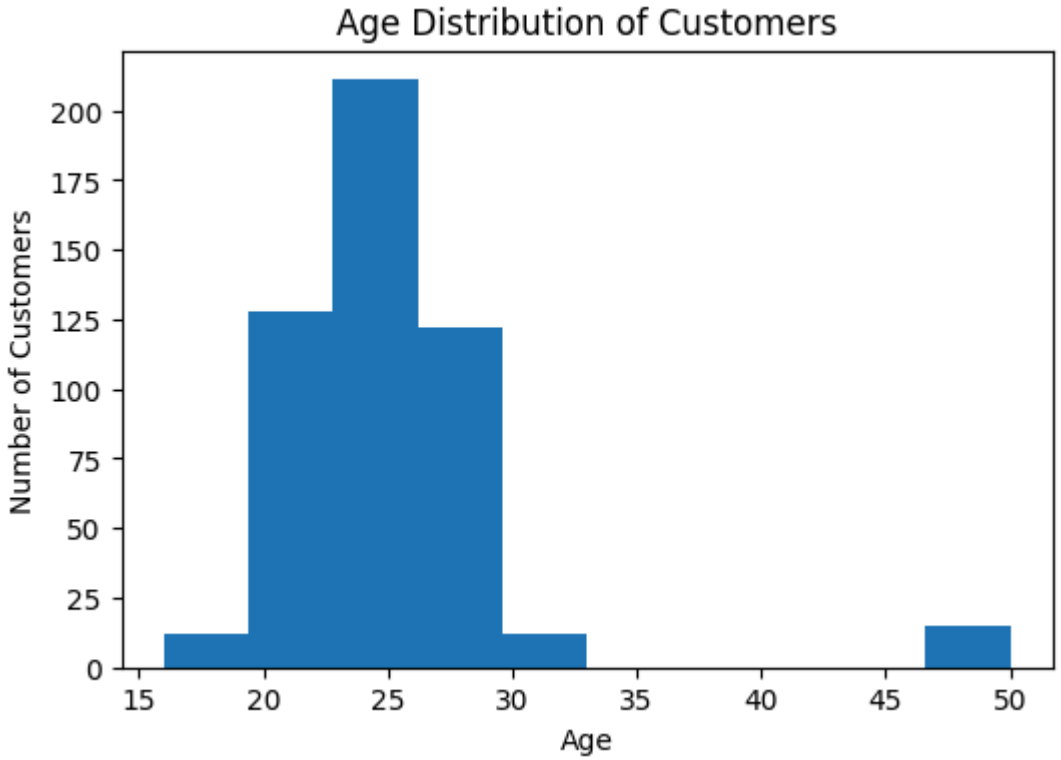
In [111…
```
# Step 1: Convert everything to string first
df['Order_Frequency'] = df['Order_Frequency'].astype(str)

# Step 2: Fix the weird dash
df['Order_Frequency'] = df['Order_Frequency'].str.replace('\x96', '-', regex=True)
```

```
# Step 3: Strip spaces
df['Order_Frequency'] = df['Order_Frequency'].str.strip()

# Step 4: Map properly
frequency_map = {
    'Rarely': 1,
    'Weekly': 2,
    '2-3 times a week': 3,
    'Daily': 4,
    '1': 1,
    '2': 2
}

df['Order_Frequency'] = df['Order_Frequency'].map(frequency_map)
```

In [112…
```
df['Order_Frequency'].dtype
```

Out[112… dtype('int64')

In [113…
```
plt.figure(figsize=(6,4))
plt.hist(df['Age'], bins=10)
plt.title("Age Distribution of Customers")
plt.xlabel("Age")
plt.ylabel("Number of Customers")
plt.show()
```



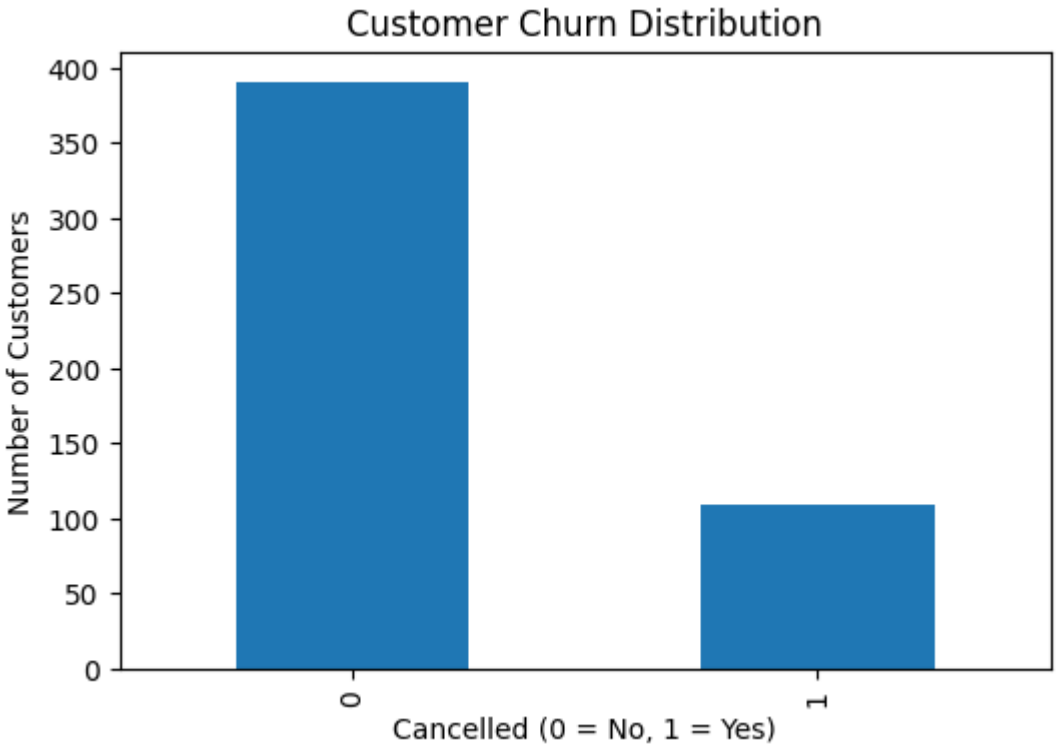
In [114…
```
df['Cancelled'] = df['Cancelled'].map({
    'Yes': 1,
    'No': 0
})
```

In [115…
```
df['Cancelled'].dtype
```

Out[115… dtype('int64')

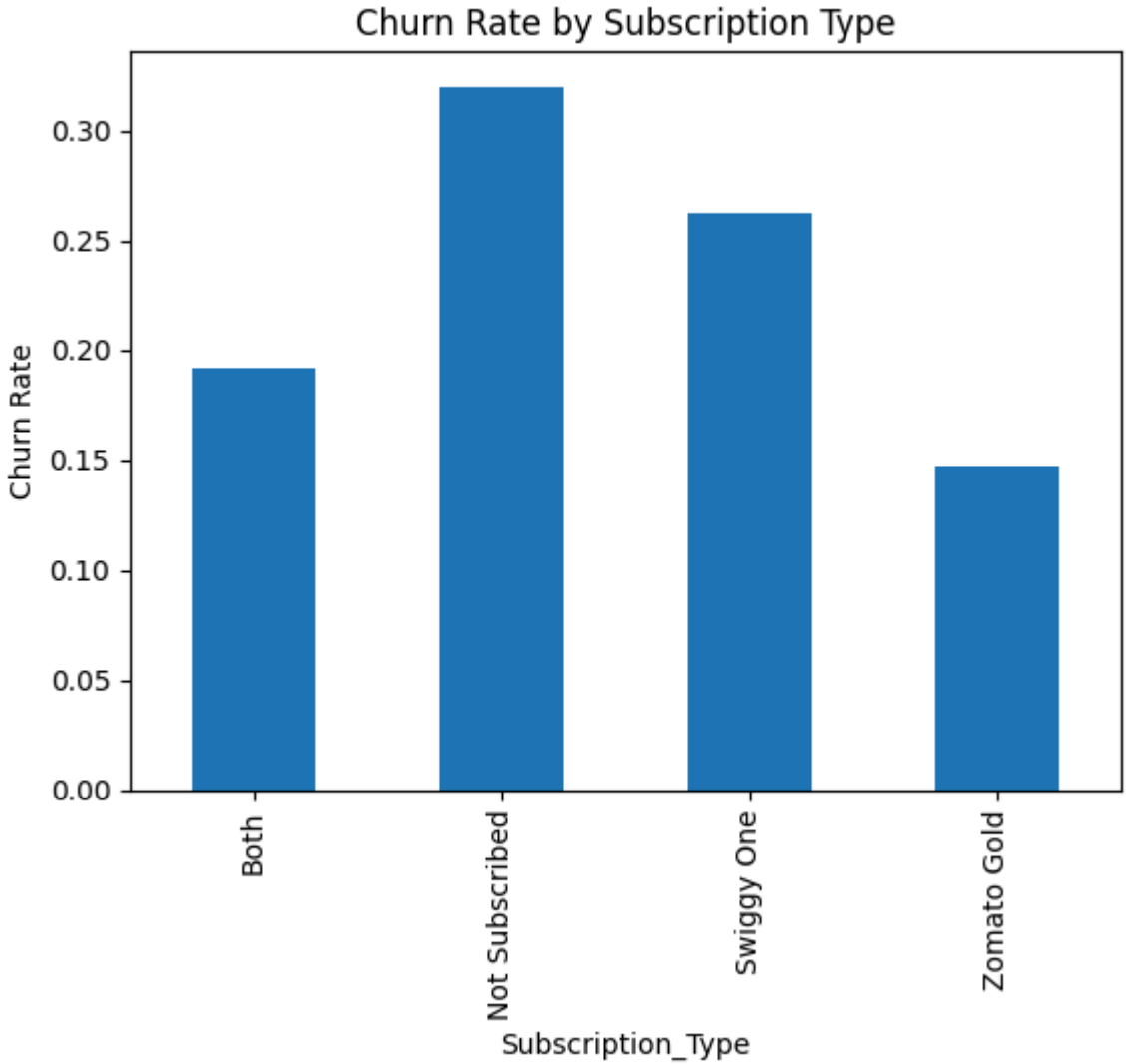
In [116…
```
plt.figure(figsize=(6,4))
df['Cancelled'].value_counts().plot(kind='bar')

plt.title("Customer Churn Distribution")
plt.xlabel("Cancelled (0 = No, 1 = Yes)")
```

```
plt.ylabel("Number of Customers")
plt.show()
```



In [117…

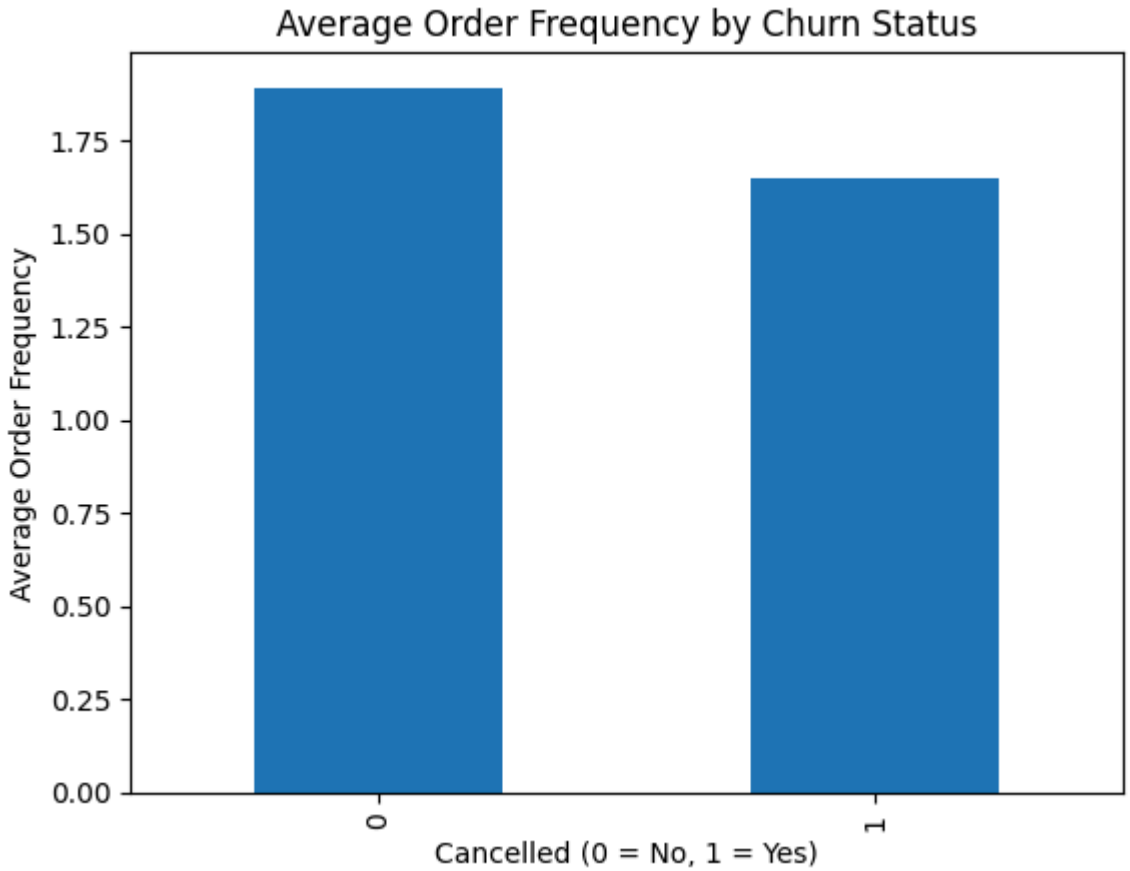
```
df.groupby('Subscription_Type')['Cancelled'].mean().plot(kind='bar')
plt.title('Churn Rate by Subscription Type')
plt.ylabel('Churn Rate')
plt.show()
```



In [118…
```
df.groupby('Cancelled')['Order_Frequency'].mean()
```

Out[118…
```
Cancelled
0    1.892583
1    1.651376
Name: Order_Frequency, dtype: float64
```

In [119…
```
df.groupby('Cancelled')['Order_Frequency'].mean().plot(kind='bar')
plt.title('Average Order Frequency by Churn Status')
plt.xlabel("Cancelled (0 = No, 1 = Yes)")
plt.ylabel('Average Order Frequency')
plt.show()
```

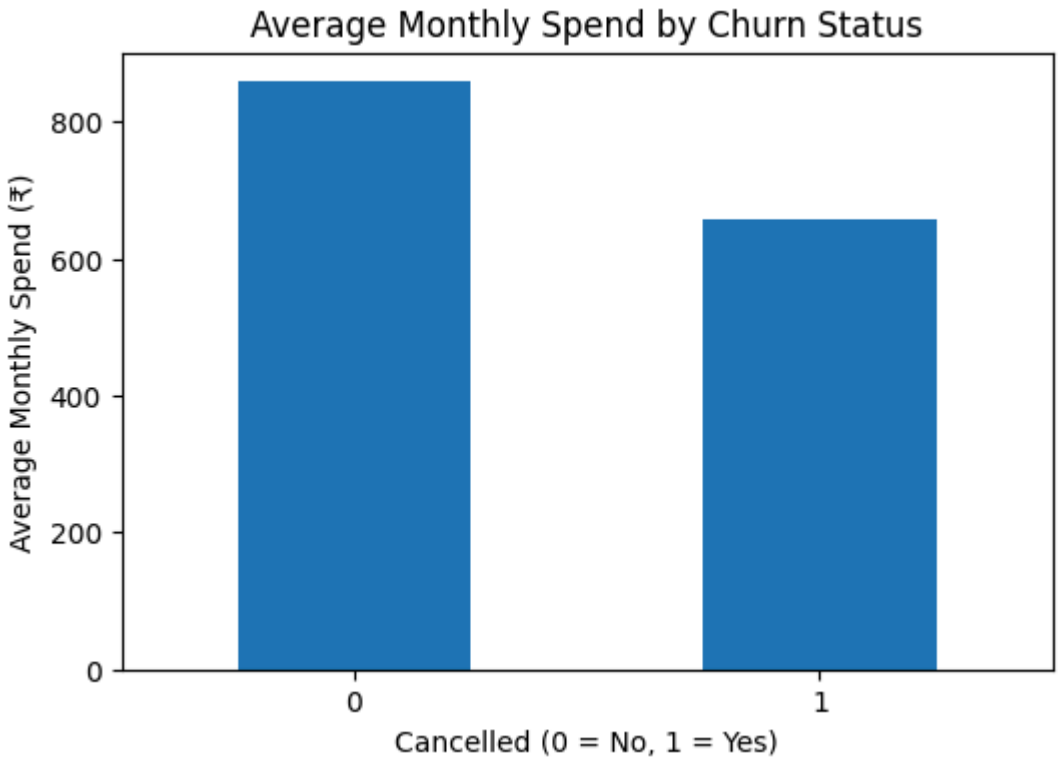


In [120…
```
avg_spend = df['Monthly_Spend'].mean()
print("Average Monthly Spend:", round(avg_spend,2))

spend_by_churn = df.groupby('Cancelled')['Monthly_Spend'].mean()
print(spend_by_churn)
```

```
Average Monthly Spend: 814.0
Cancelled
0    858.056266
1    655.963303
Name: Monthly_Spend, dtype: float64
```

In [121…

```
plt.figure(figsize=(6,4))
df.groupby('Cancelled')['Monthly_Spend'].mean().plot(kind='bar')

plt.title("Average Monthly Spend by Churn Status")
plt.xlabel("Cancelled (0 = No, 1 = Yes)")
plt.ylabel("Average Monthly Spend (₹)")
plt.xticks(rotation=0)

plt.show()
```

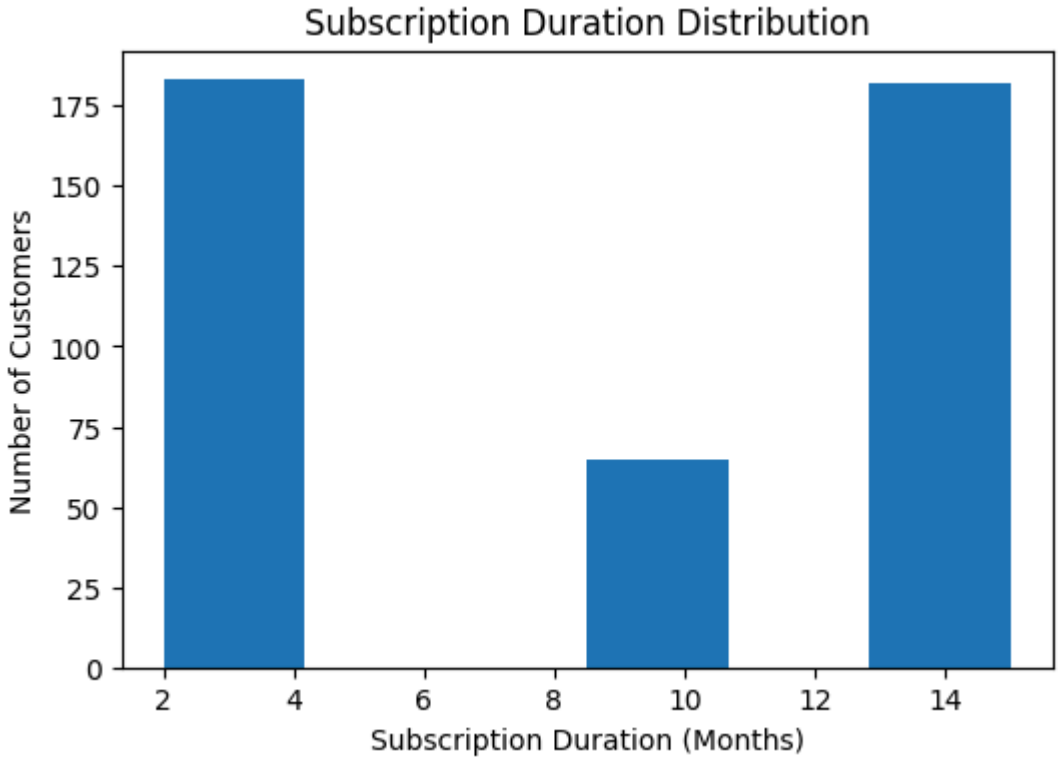


In [122…
```
duration_map = {
    "Less than 3 months": 2,
    "3–6 months": 4.5,
    "3-6 months": 4.5,
    "6-12 months": 9,
    "More than 1 year": 15
}

df["Subscription_Duration_Months"] = df["Subscription_Duration"].map(duration_map)
```

In [123…
```
avg_duration = df["Subscription_Duration_Months"].mean()
print("Average Subscription Duration:", round(avg_duration,2))
```

```
Average Subscription Duration: 8.56
```

In [124…
```
plt.figure(figsize=(6,4))

df["Subscription_Duration_Months"].plot(kind="hist", bins=6)

plt.title("Subscription Duration Distribution")
plt.xlabel("Subscription Duration (Months)")
plt.ylabel("Number of Customers")

plt.show()
```

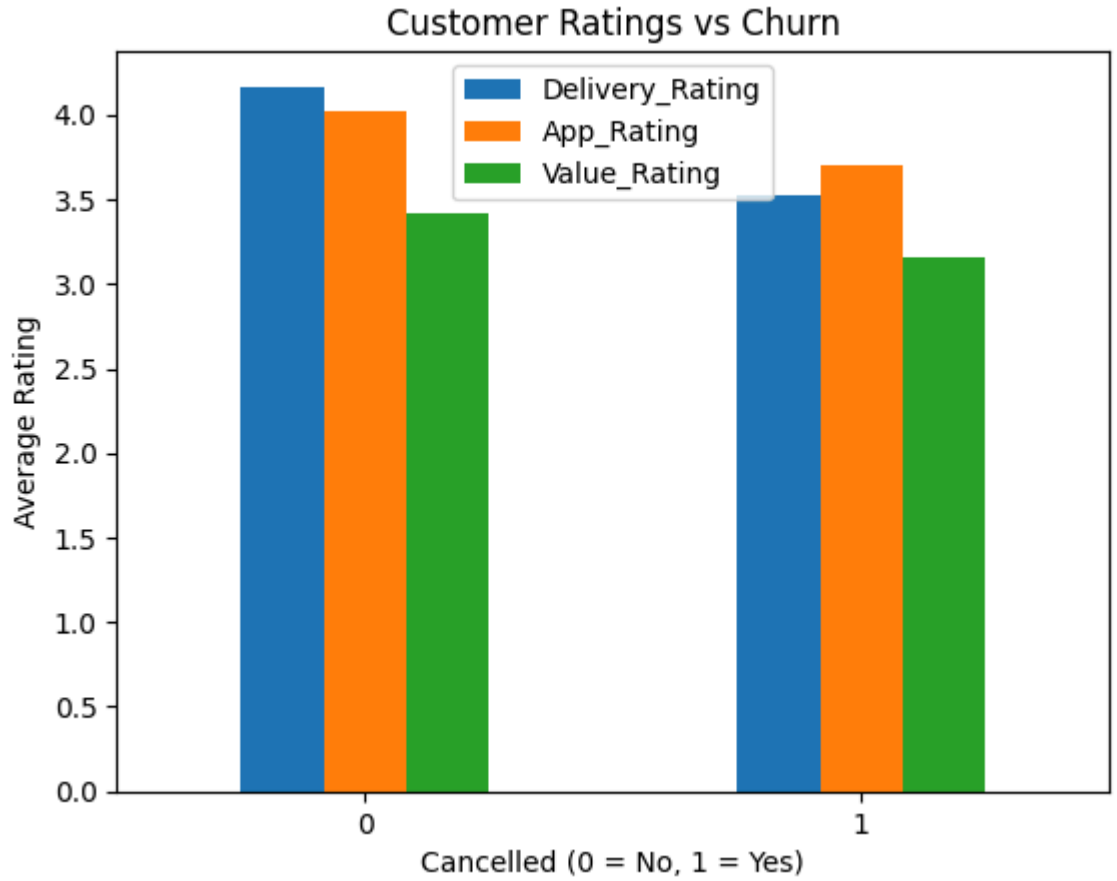


In [125…
```
print("Average Delivery Rating:", round(df['Delivery_Rating'].mean(),2))
print("Average App Rating:", round(df['App_Rating'].mean(),2))
print("Average Value Rating:", round(df['Value_Rating'].mean(),2))
```

```
Average Delivery Rating: 4.03
Average App Rating: 3.96
Average Value Rating: 3.36
```

In [126…
```
plt.figure(figsize=(6,4))
df.groupby('Cancelled')[['Delivery_Rating','App_Rating','Value_Rating']].mean().plot(kind='bar')
plt.title("Customer Ratings vs Churn")
plt.xlabel("Cancelled (0 = No, 1 = Yes)")
plt.ylabel("Average Rating")
plt.xticks(rotation=0)
plt.show()
```

```
<Figure size 600x400 with 0 Axes>
```

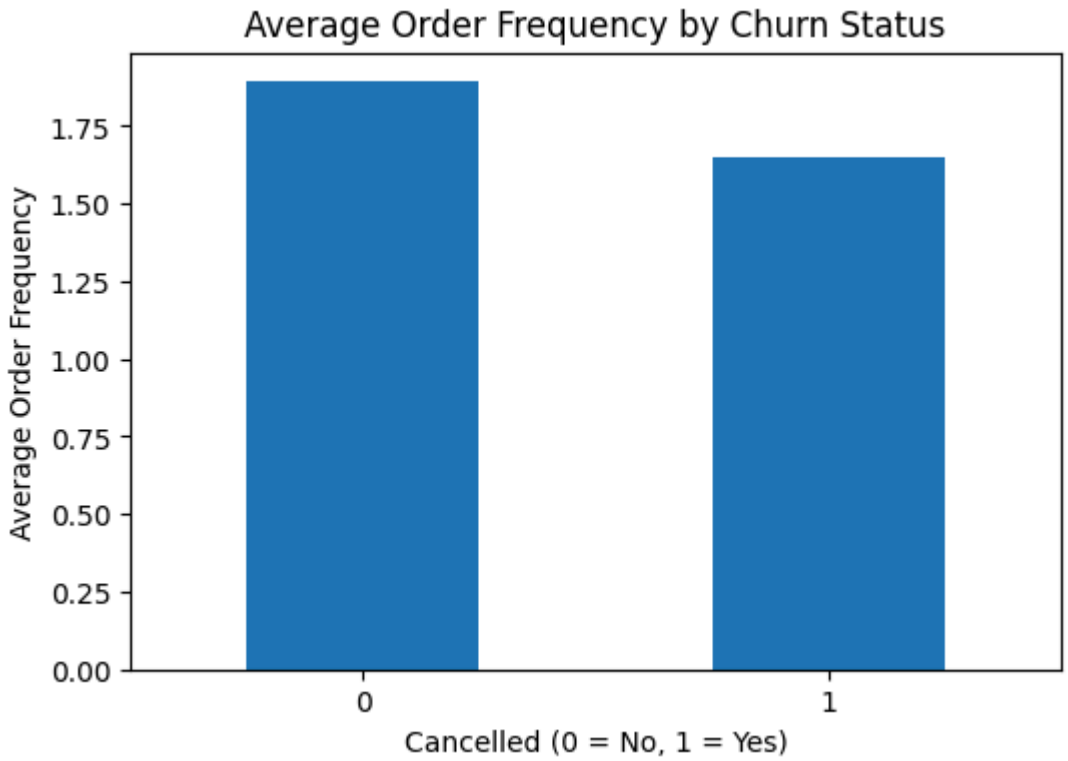


In [127…
```
order_freq = df.groupby('Cancelled')['Order_Frequency'].mean()
print(order_freq)
```

```
Cancelled
0    1.892583
1    1.651376
Name: Order_Frequency, dtype: float64
```

In [128…
```
plt.figure(figsize=(6,4))

df.groupby('Cancelled')['Order_Frequency'].mean().plot(kind='bar')

plt.title("Average Order Frequency by Churn Status")
```

```
plt.xlabel("Cancelled (0 = No, 1 = Yes)")
plt.ylabel("Average Order Frequency")
plt.xticks(rotation=0)

plt.show()
```

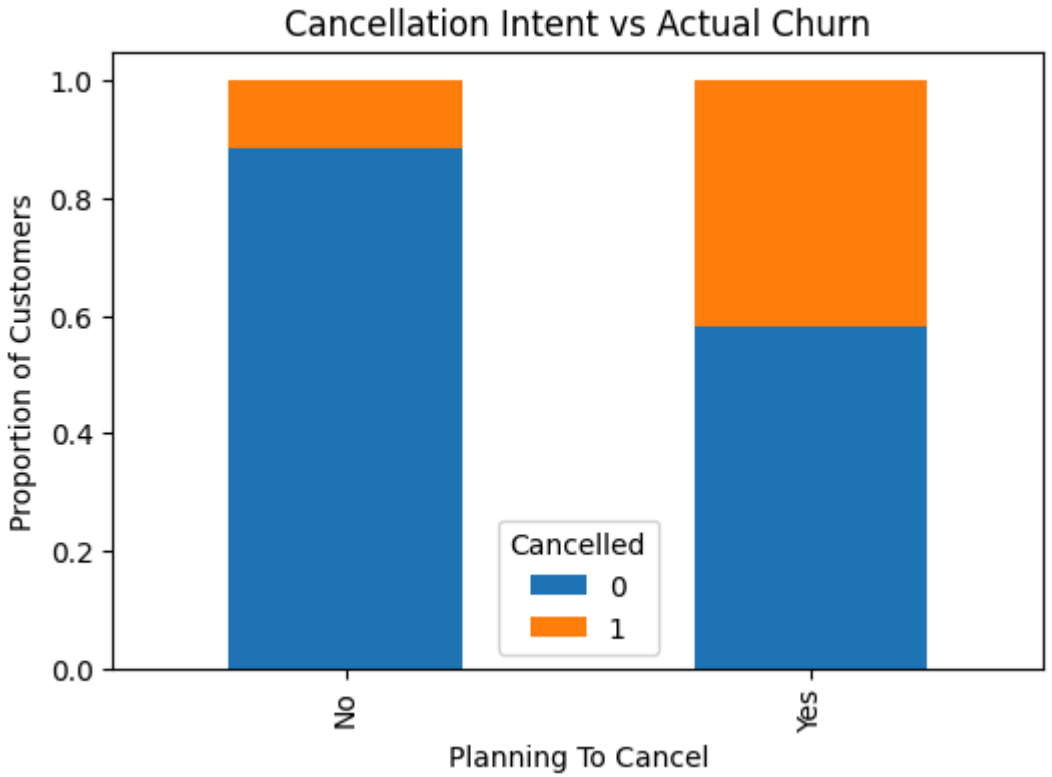


In [129…

```
intent_table = pd.crosstab(
    df['Planning_To_Cancel'],
    df['Cancelled'],
    normalize='index'
)

print(intent_table)
```

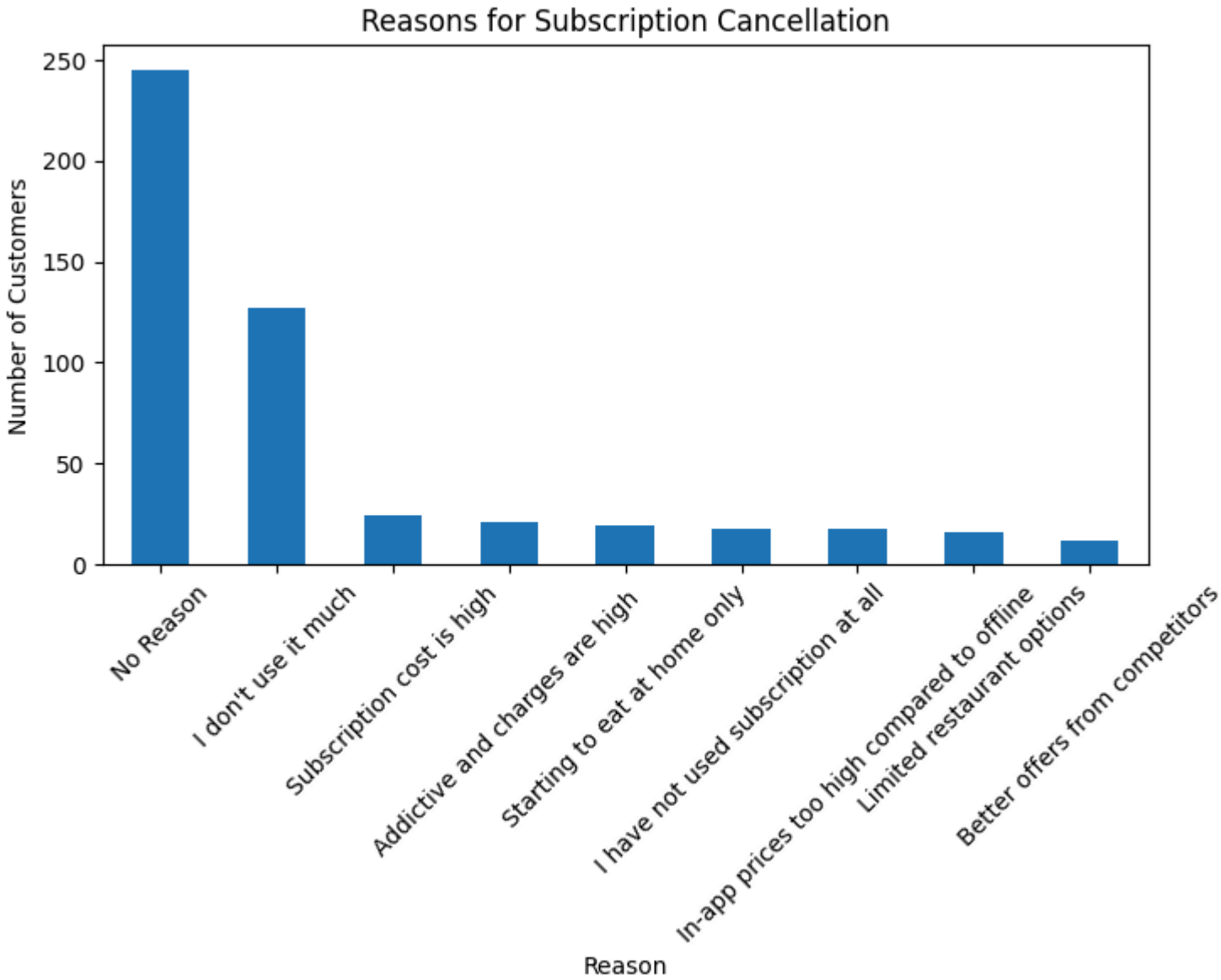
```
Cancelled                  0         1
Planning_To_Cancel
No                  0.884848  0.115152
Yes                 0.582353  0.417647
```

In [130…
```
intent_table.plot(kind='bar', stacked=True, figsize=(6,4))

plt.title("Cancellation Intent vs Actual Churn")
plt.xlabel("Planning To Cancel")
plt.ylabel("Proportion of Customers")

plt.show()
```



In [131…
```
plt.figure(figsize=(8,4))
df['Cancellation_Reason'].value_counts().plot(kind='bar')
plt.title("Reasons for Subscription Cancellation")
```

```
plt.xlabel("Reason")
plt.ylabel("Number of Customers")
plt.xticks(rotation=45)
plt.show()
```



In [157…
```
df.isna().sum()
```

Out[157…
```
Age                           0
City                          0
Occupation                    0
Subscription_Type             0
Subscription_Duration         0
Order_Frequency               0
Monthly_Spend                 0
Delivery_Rating               0
Restaurant_Rating             0
App_Rating                    0
Value_Rating                  0
Cancelled                     0
Planning_To_Cancel            0
Cancellation_Reason           0
Recommendation_Score          0
Renew_Subscription            0
Subscription_Duration_Months    70
dtype: int64
```

In [161…
```
df.drop(columns=["Subscription_Duration_Months"], inplace=True)
```

In [163…
```
df.head()
```

Out[163...

	Age	City	Occupation	Subscription_Type	Subscription_Duration	Order_Frequency	Monthly_Spend	Delivery_Rating	Restaurant_I
0	21	Thane	Student	Zomato Gold	More than 1 year	2	1500	4	
1	27	Thane	Working Professional	Zomato Gold	More than 1 year	2	0	4	
2	21	Ulhasnagar	Student+ working	Both	More than 1 year	3	1500	5	
3	21	Mumbai	Student	Zomato Gold	Less than 3 months	2	500	4	
4	21	Mumbai	Student	Not Subscribed	Less than 3 months	2	500	4	

In [171...

```
y = df["Cancelled"]
X = df.drop("Cancelled", axis=1)
```

In [173...

```
X = pd.get_dummies(X, drop_first=True)
```

In [175...

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [177...

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [179...

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    max_iter=2000,
    class_weight="balanced"
)
model.fit(X_train, y_train)
```

```
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)

y_pred = model.predict(X_test)

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.79
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.84      0.86        75
           1       0.57      0.64      0.60        25

    accuracy                           0.79       100
   macro avg       0.72      0.74      0.73       100
weighted avg       0.80      0.79      0.79       100

Confusion Matrix:
[[63 12]
 [ 9 16]]
```

In [181…

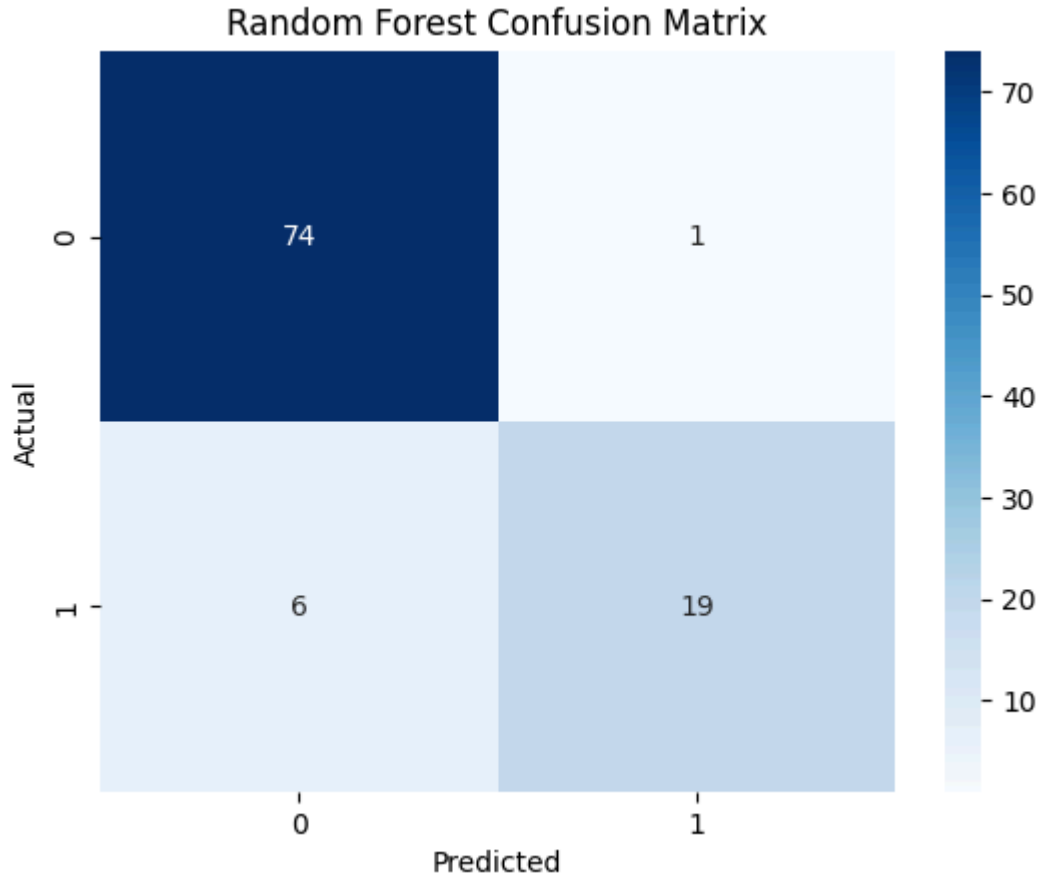
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=12,
    random_state=42
)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.93

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.99      0.95        75
           1       0.95      0.76      0.84        25

    accuracy                           0.93       100
   macro avg       0.94      0.87      0.90       100
weighted avg       0.93      0.93      0.93       100


Confusion Matrix:
 [[74  1]
 [ 6 19]]
```

In [182…

```
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest Confusion Matrix")
plt.show()
```

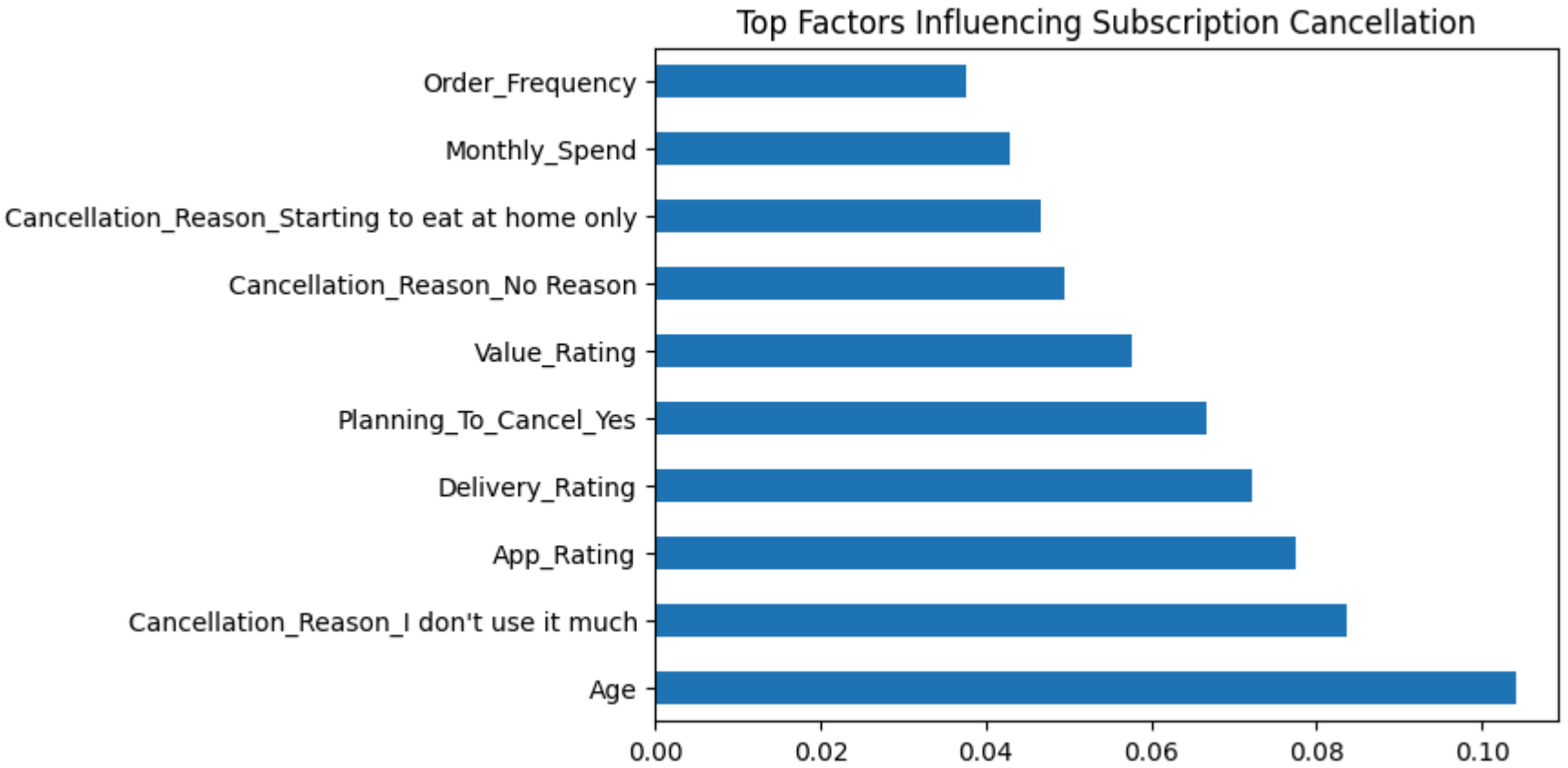


In [185…
```
print("Logistic Regression Accuracy:", accuracy)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Logistic Regression Accuracy: 0.79
Random Forest Accuracy: 0.93
```

In [187…
```
import matplotlib.pyplot as plt
import pandas as pd

importance = rf.feature_importances_
features = X.columns

importance_df = pd.Series(importance, index=features)
```

```
importance_df.nlargest(10).plot(kind="barh")
plt.title("Top Factors Influencing Subscription Cancellation")
plt.show()
```



In [191…
```
df.groupby('Cancelled')['Recommendation_Score'].mean()
```

Out[191…
```
Cancelled
0    3.514066
1    3.229358
Name: Recommendation_Score, dtype: float64
```

In [193…
```
pd.crosstab(df['Planning_To_Cancel'], df['Cancelled'], normalize='index')
```

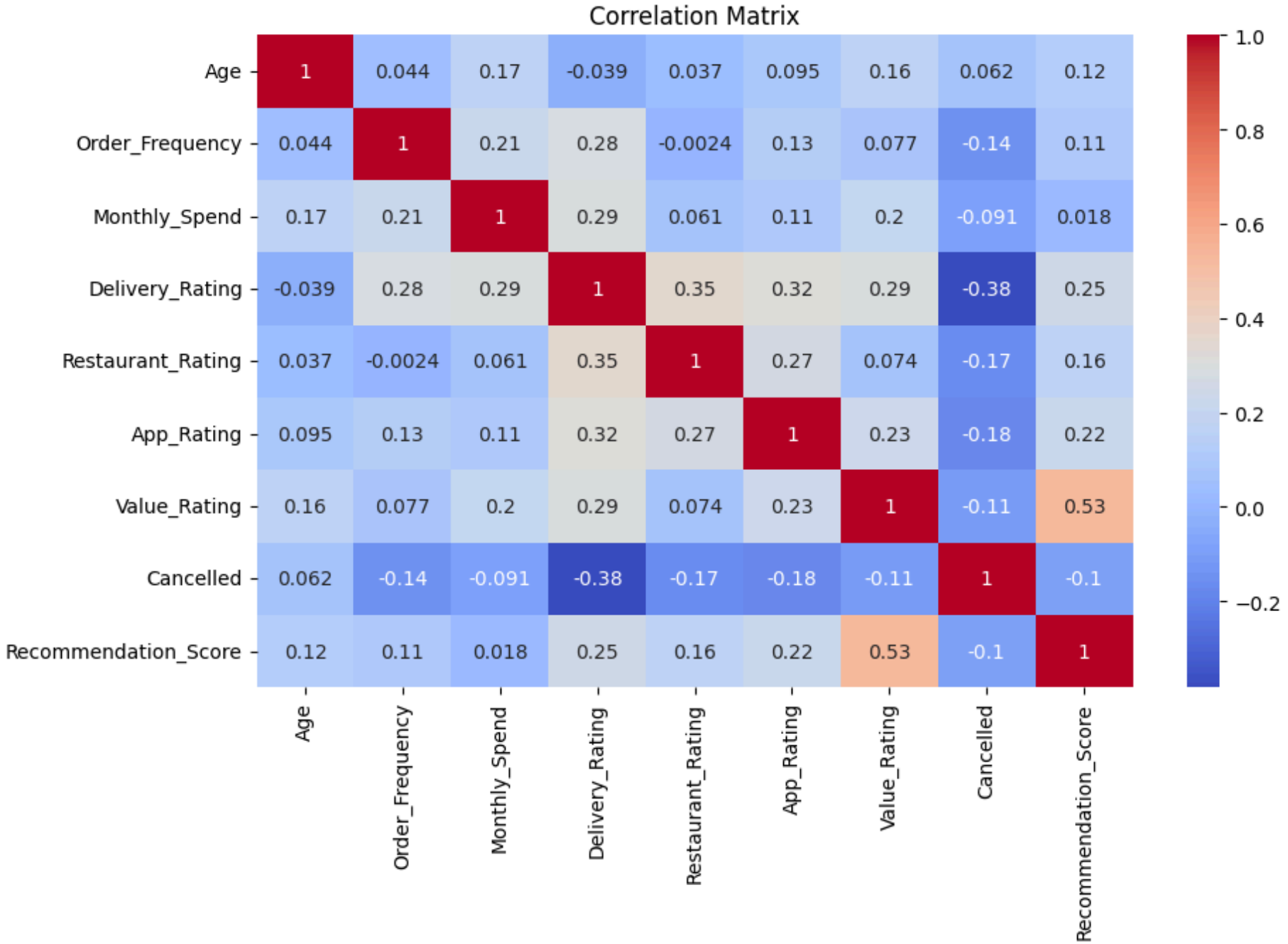
Out[193…

Cancelled	0	1
Planning_To_Cancel		
No	0.884848	0.115152
Yes	0.582353	0.417647

In [195…
```
df.groupby('Occupation')['Cancelled'].mean().sort_values(ascending=False)
```

Out[195…
```
Occupation
Working Professional              0.337500
Student                           0.173759
Student + working professional    0.136364
Student+ working                  0.136364
Business                          0.000000
Name: Cancelled, dtype: float64
```

In [197…
```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```


Correlation Matrix
Age 1 0.044 0.17 -0.039 0.037 0.095 0.16 0.062 0.12
Order_Frequency 0.044 1 0.21 0.28 -0.0024 0.13 0.077 -0.14 0.11
Monthly_Spend 0.17 0.21 1 0.29 0.061 0.11 0.2 -0.091 0.018
Delivery_Rating -0.039 0.28 0.29 1 0.35 0.32 0.29 -0.38 0.25
Restaurant_Rating 0.037 -0.0024 0.061 0.35 1 0.27 0.074 -0.17 0.16
App_Rating 0.095 0.13 0.11 0.32 0.27 1 0.23 -0.18 0.22
Value_Rating 0.16 0.077 0.2 0.29 0.074 0.23 1 -0.11 0.53
Cancelled 0.062 -0.14 -0.091 -0.38 -0.17 -0.18 -0.11 1 -0.1
Recommendation_Score 0.12 0.11 0.018 0.25 0.16 0.22 0.53 -0.1 1
Age
Order_Frequency
Monthly_Spend
Delivery_Rating
Restaurant_Rating
App_Rating
Value_Rating
Cancelled
Recommendation_Score
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

In [199…
```
df['Churn_Probability'] = model.predict_proba(X)[:,1]

df['Risk_Level'] = pd.cut(
    df['Churn_Probability'],
    bins=[0,0.3,0.6,1],
    labels=["Low Risk","Medium Risk","High Risk"]
)

print(df[['Churn_Probability','Risk_Level']].head())
```

```
   Churn_Probability Risk_Level
0           1.000000  High Risk
1           0.990709  High Risk
2           1.000000  High Risk
3           0.999961  High Risk
4           0.999980  High Risk
```

```
C:\Users\NIKITA\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2732: UserWarning: X has feature names, but LogisticReg
ression was fitted without feature names
  warnings.warn(
```

In [203…
```
duration_map = {
    "Less than 3 months": 2,
    "3–6 months": 4.5,
    "3-6 months": 4.5,
    "6-12 months": 9,
    "More than 1 year": 15
}

df["Subscription_Duration_Months"] = df["Subscription_Duration"].map(duration_map)
```

In [205…
```
avg_duration = df["Subscription_Duration_Months"].mean()
print("Average Subscription Duration:", round(avg_duration,2))
```

```
Average Subscription Duration: 8.56
```

In [207…
```
churn_rate = df['Cancelled'].mean() * 100
retention_rate = 100 - churn_rate
avg_spend = df['Monthly_Spend'].mean()
avg_rating = df['App_Rating'].mean()
avg_duration = df["Subscription_Duration_Months"].mean()
```

```
avg_delivery = df['Delivery_Rating'].mean()
avg_app = df['App_Rating'].mean()
avg_value = df['Value_Rating'].mean()

print("Churn Rate:", round(churn_rate,2), "%")
print("Retention Rate:", round(retention_rate,2), "%")
print("Average Monthly Spend:", round(avg_spend,2))
print("Average App Rating:", round(avg_rating,2))
print("Average Subscription Duration:", round(avg_duration,2), "months")
print("Average Delivery Rating:", round(avg_delivery,2))
print("Average App Rating:", round(avg_app,2))
print("Average Value Rating:", round(avg_value,2))
```

```
Churn Rate: 21.8 %
Retention Rate: 78.2 %
Average Monthly Spend: 814.0
Average App Rating: 3.96
Average Subscription Duration: 8.56 months
Average Delivery Rating: 4.03
Average App Rating: 3.96
Average Value Rating: 3.36
```

In [ ]: